ONBRD-07: Understanding Your Data

> **Series:** ONBRD — Dynatrace Onboarding | **Notebook:** 7 of 10 | **Created:** December 2025 | **Last Updated:** 07/24/2026

Exploring What Dynatrace Discovered

With OneAgent deployed, Dynatrace has automatically discovered your infrastructure, processes, and services. This notebook helps you understand what's been found and how to explore your data.

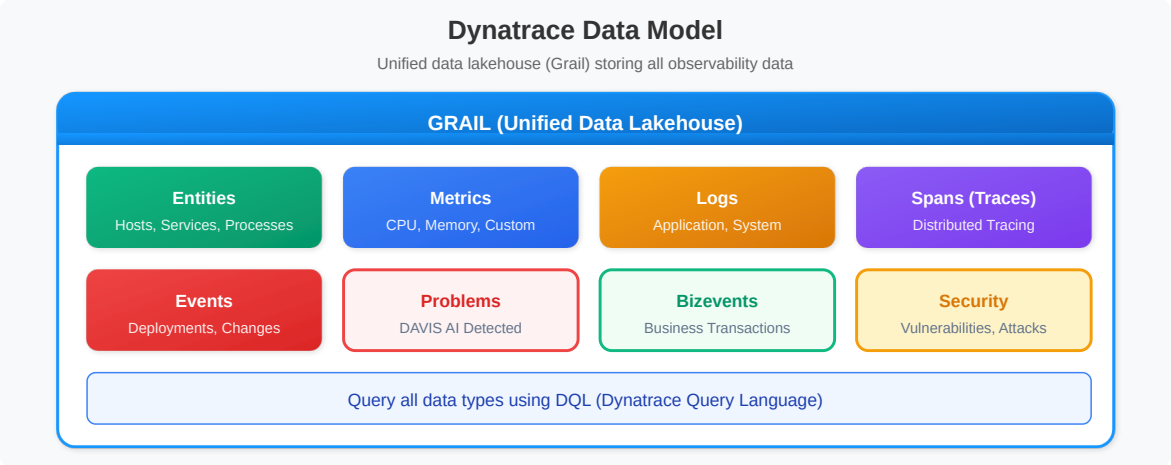
Table of Contents

Prerequisites

- OneAgent deployed on at least one host (ONBRD-05)
- Environment organized with tags (ONBRD-06)
- 15-30 minutes elapsed since deployment for full discovery
- DQL query permissions

1. The Dynatrace Data Model

Dynatrace organizes data into a unified model:

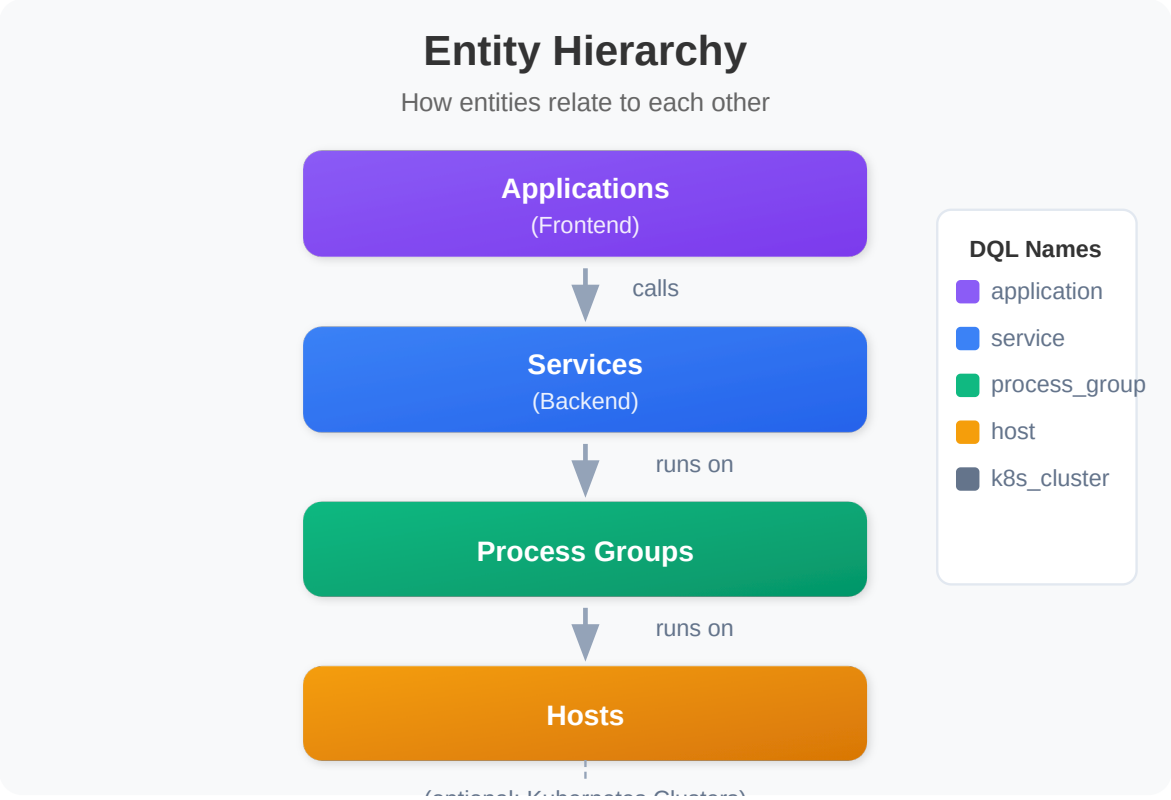


Key Concepts

Concept	Description	Example
Entity	Any monitored component	Host, Service, Process
Metric	Numeric measurement over time	CPU usage, response time
Log	Textual event record	Application log entry
Span	Single operation in a trace	HTTP request, DB query
Event	Point-in-time occurrence	Deployment, config change
Problem	DAVIS-detected issue	Service slowdown

2. Entities and Relationships

Dynatrace automatically discovers and relates entities:



Common Entity Types

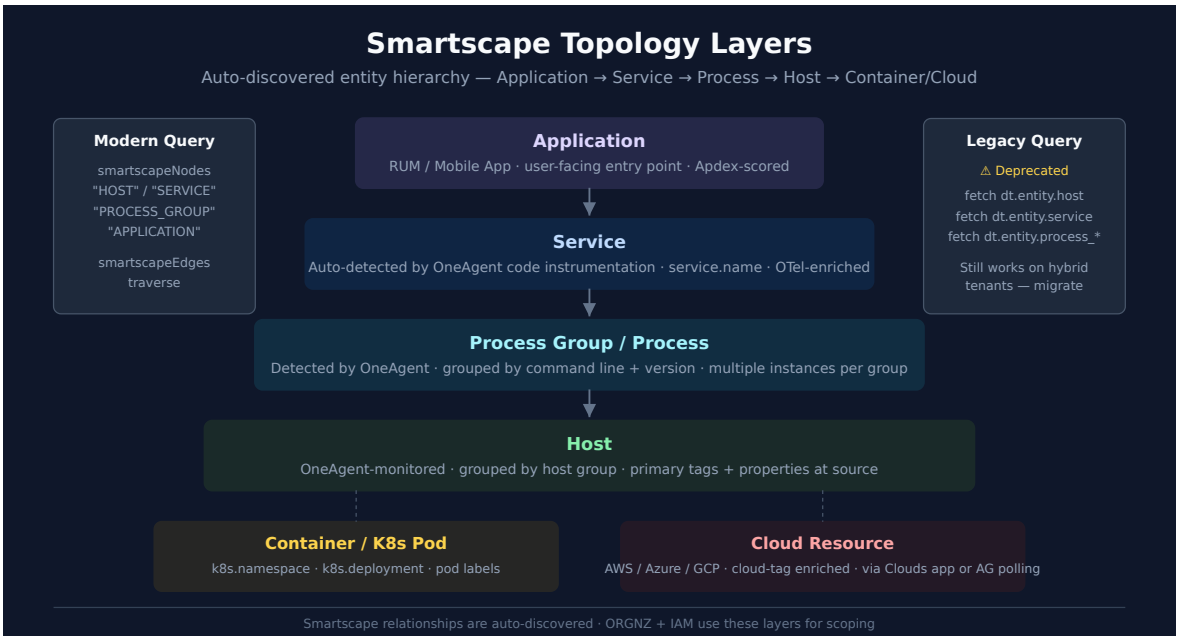
Entity Type	DQL Name	What It Represents
Host	`dt.entity.host`	Physical or virtual machine
Process Group	`dt.entity.process_group`	Set of identical processes
Service	`dt.entity.service`	Logical backend service
Application	`dt.entity.application`	Frontend application
K8s Cluster	`dt.entity.kubernetes_cluster`	Kubernetes cluster
K8s Namespace	`dt.entity.cloud_application_namespace`	K8s namespace
K8s Workload	`dt.entity.cloud_application`	Deployment, DaemonSet, etc.

3. Exploring Topology

The topology view shows the visual representation of your environment.

Location: Infrastructure app → Hosts → Select any host → Dependencies

Understanding Topology Layers



Layer	Shows	Use Case
Applications	Frontend apps, user sessions	User experience
Services	Backend services, APIs	Service dependencies
Processes	Running processes	Process mapping
Hosts	Servers, VMs, containers	Infrastructure view

Reading Topology Views

- **Nodes** = Entities
- **Lines** = Relationships (calls, runs on)
- **Line thickness** = Traffic volume
- **Colors** = Health status (green/yellow/red)

Navigation Tips

- Click any entity to see details
- Use filters to focus on specific services
- Hover over connections to see traffic metrics
- Use the Services app for service-to-service dependencies

4. Data Types in Grail

Grail stores different data types — each queried via a specific DQL command. Retention is **per-bucket and customer-configurable**, not a fixed default.

Data Type	DQL Fetch / Command	Typical Use
Logs	`fetch logs`	Troubleshooting, audit, compliance
Spans	`fetch spans`	Distributed tracing
Metrics	`timeseries` *(not `fetch`)*	Performance monitoring, SLOs
Events	`fetch events`	Change tracking, infra events
Bizevents	`fetch bizevents`	Business transactions, conversion funnels
Davis problems	`fetch dt.davis.problems`	Detected incidents (uses `event.status` / `event.end` fields)
Davis events	`fetch dt.davis.events`	Raw signals that feed problem detection (kind = `DAVIS_EVENT`)
Security events	`fetch securityEvents`	Vulnerabilities, security signals
RUM sessions	`fetch usersessions`	Session-level RUM aggregates
RUM individual events	`fetch user.events`	Page views, clicks, requests, errors
RUM session replays	`fetch user.replays`	Recorded session replays

Data Type	DQL Fetch / Command	Typical Use
Entities (topology)	`smartscapeNodes "<TYPE>"` *(modern)* / `fetch dt.entity.<type>` *(legacy)*	Topology queries; `dt.entity.*` is deprecated, prefer `smartscapeNodes` for new queries

Data Retention

Retention is **bucket-scoped and customer-configurable** — there is no universal default. Out-of-the-box retention varies by license tier and Grail bucket configuration. Inspect your tenant's bucket retention with:

```
// List configured Grail buckets and their retention (use Bucket Management UI for full detail)
fetch dt.system.buckets
| fields name, table, retention
| sort name asc
```

Typical starting points (subject to customer configuration):

Data Type	Common Out-of-the-Box Retention
Metrics (raw)	14 days
Metrics (aggregated)	up to 5 years
Logs	35 days (default bucket; can be 14d / 90d / 365d per tier)
Spans	14 days
Events / Bizevents	35 days
Davis problems	35 days

> **Where to go deeper:**
> - **ORGNZ-02 / ORGNZ-99** — Grail bucket strategy and retention design
> - **OPLOGS series** — Log processing in OpenPipeline
> - **OPMIG series** — Classic Logs → OpenPipeline migration
> - **OPIPE series** — OpenPipeline beyond logs (spans, metrics, events, bizevents)
> - **SPANS series** — Distributed tracing and span analysis

5. Discovery Queries

Use these queries to understand what Dynatrace has discovered in your environment.

Infrastructure Discovery

```
// Count all entity types in your environment
fetch dt.entity.host | summarize hosts = count()
// Run separately for other types:
// fetch dt.entity.service | summarize services = count()
// fetch dt.entity.process_group | summarize process_groups = count()

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST" | summarize hosts = count()
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
```

```
// List all hosts with key details
fetch dt.entity.host
| fields entity.name,
         state,
         monitoringMode,
         osType,
         cpuCores,
         physicalMemory
| sort entity.name
| limit 100

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only fields state / monitoringMode, which have NO Smartscape node
equivalent
// (Smartscape expresses liveness via node lifetime, not a state field). Keep
the classic
// query above for state / monitoring-mode detail.
// Other fields do map: osType -&gt; os.type (LINUX -&gt; OS_TYPE_LINUX);
cpuCores -&gt; cores; physicalMemory -&gt; host.physical.memory; entity.name -
&gt; name.
```

```
// Group hosts by OS type
fetch dt.entity.host
| summarize count = count(), by: {osType}
| sort count desc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | summarize count = count(), by: {os.type}
//   | sort count desc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: osType -&gt; os.type (LINUX -&gt; OS_TYPE_LINUX).
```

Service Discovery

```
// List all discovered services
fetch dt.entity.service
| fields entity.name, serviceType
| sort entity.name
| limit 100

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | fields name, dt.service.sdv1_type
//   | sort name
//   | limit 100
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: serviceType -&gt; dt.service.sdv1_type; entity.name -&gt; name.
```

```
// Group services by type
fetch dt.entity.service
| summarize count = count(), by: {serviceType}
| sort count desc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | summarize count = count(), by: {dt.service.sdv1_type}
//   | sort count desc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: serviceType -&gt; dt.service.sdv1_type.
```

```
// Find services with recent traffic (spans)
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize request_count = count(), by: {service.name}
| sort request_count desc
| limit 20
```

Process Discovery

```
// List process groups
fetch dt.entity.process_group
| fields entity.name
| sort entity.name
| limit 50

// Smartscape note (dt.entity.* is deprecated but still functional):
Smartscape models
// individual processes, not process GROUPS — smartscapeNodes "PROCESS" is a
different
// granularity (process instances), so its results are not comparable to a
process-group
// query. Keep the classic dt.entity.process_group query above.
```

```
// Count process groups
fetch dt.entity.process_group
| summarize count = count()

// Smartscape note (dt.entity.* is deprecated but still functional):
Smartscape models
// individual processes, not process GROUPS — smartscapeNodes "PROCESS" is a
different
// granularity (process instances), so its results are not comparable to a
process-group
// query. Keep the classic dt.entity.process_group query above.
```

Log Discovery

Coverage: what was discovered vs. ingested

> **Coverage vs. volume — the log-module self-monitoring events.** During onboarding, confirm not just *what volume* is arriving but *what OneAgent discovered and whether each source is actually being ingested*. The log module emits a `log_source.status` event per discovered source into `dt.system.events`, carrying `log.source.file_status` and `log.source.ingest_status` — so it reveals sources OneAgent **detected but is not ingesting**, which a `fetch logs` count cannot show (no stored records means nothing to count). It scans events, not raw logs, and needs only event-read.
>
> This event stream is **Early Access** and requires **opt-in** (the OneAgent log module must be enabled to send self-monitoring events; its content folds into the built-in *Log ingest Overview* dashboard for Dynatrace **1.339+** — verify it is flowing in your tenant before relying on it). See **FAQ-08** (Recommended Approach) for the full file-status × ingest-status coverage matrix and the `fetch logs` fallback for tenants without the opt-in.

```
// Log-source coverage from the OneAgent log-module self-monitoring events.
// Unlike `fetch logs` (which sees only sources that produced records), this
// surfaces sources OneAgent DETECTED but is NOT ingesting. Scans events, not
raw logs.
fetch dt.system.events, from: now() - 24h
| filter event.provider == "Log Module" and event.type == "log_source.status"
| dedup event.id, sort:{ timestamp desc }
| filter event.status == "Active"
| fieldsAdd coverage =
    if(log.source.file_status == "FILE_STATUS_OK" and log.source.ingest_status
== "Ingested", "Available | Ingesting", else:
    if(log.source.file_status == "FILE_STATUS_OK" and log.source.ingest_status
== "Not ingested", "Available | NOT ingesting", else:
    if(log.source.ingest_status == "Partially ingested", "Partial ingestion",
    else: "Unsupported or issue")))
| summarize sources = count(), by:{coverage}
| sort sources desc
```

```
// Check log volume by source
fetch logs, from: now() - 1h
| summarize log_count = count(), by: {log.source}
| sort log_count desc
| limit 20
```

```
// Check log volume by severity
fetch logs, from: now() - 1h
| summarize log_count = count(), by: {loglevel}
| sort log_count desc
```

```
// Sample recent logs
fetch logs, from: now() - 15m
| fields timestamp, loglevel, log.source, content
| sort timestamp desc
| limit 25
```

Kubernetes Discovery (if applicable)

```
// List Kubernetes clusters
fetch dt.entity.kubernetes_cluster
| fields entity.name
| sort entity.name

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_CLUSTER"
//   | fields name
//   | sort name
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: entity.name -&gt; name.
```

```
// List namespaces
fetch dt.entity.cloud_application_namespace
| fields entity.name
| sort entity.name
| limit 50

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_NAMESPACE"
//   | fields name
//   | sort name
//   | limit 50
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: entity.name -&gt; name.
```

```
// List workloads (deployments, etc.)
fetch dt.entity.cloud_application
| fields entity.name
| sort entity.name
| limit 50

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_DEPLOYMENT"
//   | fields name
//   | sort name
//   | limit 50
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: entity.name -&gt; name.
```

Problems Discovery

```
// Check for recent problems
fetch dt.davis.problems, from: now() - 7d
| fields timestamp, display_id, title, event.status, affected_entity_types
| sort timestamp desc
| limit 20
```

```
// Problem summary by status
fetch dt.davis.problems, from: now() - 30d
| summarize problem_count = count(), by: {event.status}
| sort problem_count desc
```

6. Next Steps

Now that you understand your data:

1. **ONBRD-08: Your First Queries** — Learn DQL fundamentals
2. Explore topology for dependency visualization
3. Review any detected problems
4. Plan which additional hosts to instrument

Where to Go Deeper

- **ORGNZ-02 / ORGNZ-99** — Grail bucket strategy
- **OPLOGS / OPMIG / OPIPE** — Log and OpenPipeline depth
- **SPANS series** — Distributed tracing depth
- **AIOPS series** — Davis problems, RCA, anomaly detection deep dives
- **K8S series** — Kubernetes monitoring depth

Discovery Checklist

- [ ] Hosts discovered and showing data
- [ ] Services detected and mapped
- [ ] Process groups visible
- [ ] Topology showing relationships
- [ ] Log ingestion working (if applicable)
- [ ] Kubernetes entities visible (if applicable)
- [ ] Bucket retention reviewed against expected data class

Summary

In this notebook, you learned:

- The Dynatrace data model (entities, metrics, logs, spans, events, bizevents, security, RUM)
- How entities relate to each other
- How to navigate topology views
- Different data types stored in Grail and their DQL fetch commands
- That `dt.davis.problems` uses `event.status` / `event.end` fields (not `status` / `end_time`)
- Discovery queries for infrastructure, services, and logs
- That retention is bucket-scoped and customer-configurable

References

- Entities
- Topology and Dependencies
- Grail Data Model
- Entity Types Reference
- Davis Problems App
